

EKFNet: Learning System Noise Covariance Parameters for Nonlinear Tracking

Liang Xu and Ruixin Niu , Senior Member, IEEE

Abstract—In this paper, to reduce the time and manpower to fine-tune an extended Kalman filter (EKF), we propose a new learning framework, EKFNet, for automatically estimating the best process and measurement noise covariance parameters for an EKF from real measurement data. The EKFNet is trained end-to-end by using backpropagation through time (BPTT) over the EKF. The forward operation of EKFNet is the same as the normal EKF operation which will be used during the tracking process. During the offline training, the EKFNet uses the BPTT for passing the gradient flow to each time step and optimizing the unknown noise statistic parameters. The proposed method can choose among several optimization criteria, such as maximizing the likelihood, minimizing the measurement residual error, or minimizing the posterior state estimation error either with or without the ground truth data. The proposed method's performance is demonstrated using real GPS data, which outperforms an existing method and a manually tuned EKF.

Index Terms—EKF, fine-tuning, recurrent neural network, machine learning, backpropagation through time.

I. INTRODUCTION

POPULAR nonlinear filters for state estimation include the extended Kalman filter (EKF) [1], [2], [3], the unscented Kalman filter (UKF) [4], and the particle filter (PF) [5]. Nonlinear tracking filters have been extensively employed in computer vision [6], LiDAR tracking [7], and extended object tracking [8], for diverse applications spanning from autonomous driving [9] to surveillance systems [3], [7] and air traffic control [10]. In practice, the EKF is widely used due to its simplicity and its much lower computational load than the PF and UKF. Different from the linear assumption required by the Kalman Filter, in an EKF, the motion and observation models do not have to be linear functions of the state but need to be differentiable with respect to the state. The motion model is used for predicting the object's dynamic behavior according to the current state. The

measurement model maps the state space into the measurement space. In the EKF, these nonlinear models need to be linearized, so that the Kalman filter's recursive framework can be readily utilized.

In the motion and measurement models of a non-linear filter, noise terms are used for compensating the errors made by: (1) model simplification, (2) additional states not modeled, (3) discretization error, (4) model linearization, and (5) Gaussian noise assumptions. In practice, implementing the EKF is often challenging due to the difficulty of getting a good estimate of the motion and measurement noise covariances. Much research has been performed to estimate these covariances from data. However, it takes a significant amount of time and manpower to manually fine-tune the EKF and acquire proper system noise statistics used by the filter. It is thus desirable to develop an approach that automatically finds the best noise statistics for an EKF.

A. Previous Related Work

The EKF requires knowledge of the noise statistics for uncertainty propagation and the calculation of the filter gain. However, the ground truth noise covariance matrices are generally unknown, and need to be estimated/tuned. In [11], an auto-tuning framework for a Kalman smoother by using the proximal optimization method was introduced. However, the noise covariance pair, which is good for Kalman smoother, performs poorly for Kalman filtering [12]. Different objective functions have been proposed in [12] for tuning the EKF, but the optimization method proposed is neither efficient nor guaranteed to converge. In [13], a nonlinear programming method was proposed for tuning, but it only applies to the Kalman filter (KF). Using Bayesian optimization, an approach to find the globally optimal covariance was proposed in [14]. However, it becomes computationally prohibitive for a large training data set. Bayesian probabilistic approaches were proposed in [15], [16] for online estimation of process and measurement noise parameters, in which a finite difference method was used to numerically evaluate the gradient and Hessian of the cost function. In [17] a six-step solution to filter tuning was provided, but it is only applicable to linear Kalman filters.

With the recent advancement in deep learning, machine learning based approaches become a promising solution to process time-series data, for real time classification of maneuvering aircraft [18], [19] and for nonlinear state estimation. Some state estimation methods by training the recurrent neural network

Manuscript received 27 May 2023; revised 5 March 2024; accepted 20 May 2024. Date of publication 20 June 2024; date of current version 16 July 2024. This work was supported by the Air Force Office of Scientific Research under Grant FA9550-18-1-0362 and Grant FA9550-22-1-0038. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Victor Elvira. (Corresponding author: Ruixin Niu.)

Liang Xu is with the Motional Inc., Boston, MA 02210 USA (e-mail: liang.xu@motional.com).

Ruixin Niu is with the Department of Electrical and Computer Engineering, Virginia Commonwealth University, Richmond, VA 23284 USA (e-mail: rniu@vcu.edu).

Digital Object Identifier 10.1109/TSP.2024.3417350

(RNN) have been developed in several papers [20], [21], [22], [23], [24], [25], [26], [27]. But, the focus of these publications was on learning either the relationship between the states over different time steps [22], [23], [28] or integrating unstructured measurement data to the state space [22]. In [29], a data-driven KF has been proposed, in which the uncertainty propagation steps are replaced by a neural network to calculate the Kalman filter gain. In [18], [30], [31], a RNN was developed to dynamically adjust the process noise covariance in a Kalman filter for maneuvering target tracking, in which the focus is to adapt the process noise covariance, but not the measurement noise covariance.

Recently, a trend in data-driven particle filter is emerging to learn the motion model and measurement model also through RNNs [28], [32], [33], [34], [35], [36]. Differentiable particle filters (DPFs) were developed in [32], [33], which use backpropagation to learn the motion and measurement models and noise covariance matrices [28]. However, since the particle filter is not differentiable between time steps, the DPFs have to either use finite difference with much higher computational complexity [33] or only train the filter based on a single time step from t to $t + 1$, not end to end [32]. In the DPFs publications, both the motion and measurement models are assumed to be unknown, which need to be learned from the data.

In many system tracking problems, e.g. target tracking in a radar system, both the measurement and motion models are well known, which can be easily constructed by physical modeling and do not have to be learned from data. In such problems, using the computational power to learn the motion and measurement models is wasteful and not guaranteed to converge. Further, the learned motion and measurement models can be easily over-fitted against the training data when one has to learn in a high-dimensional hypothesis space with limited datasets. Therefore, in this paper, our research is focused on learning the best noise statistics for an EKF with given motion and measurement models.

B. Contributions

As discussed earlier, in many tracking problems, both the state and measurement models are already known. Even though sometimes in these system models, approximations have been used to convert physical models to mathematical equations, they can still precisely characterize the system. Furthermore, the system noise in the model can be used to account for/absorb the approximation errors. In the proposed EKFNet, instead of learning the system models, the statistic parameters used by the state and measurement models are learned based on the backpropagation through time (BPTT) approach [37]. BPTT is used to drive the gradients backward, to optimize the EKF's performance. Compared with the conference version of the paper [38], we have added the complete and detailed BPTT derivations and more extensive numerical results to demonstrate the effectiveness of the proposed EKFNet. In summary, the new contributions in this paper are:

- We construct a new framework for the EKF to learn the noise covariance parameters with or without ground truth data, based on BPTT.

- We efficiently drive the gradient backward, and all the gradients are calculated by computation graphs.
- We provide four different loss functions for learning the noise covariance parameters, which can be combined or used individually.
- We evaluate the proposed method using real GPS data for vehicle tracking.

The rest of this paper is organized as follows. In Section II, the EKFNet forward operation and the loss functions are described. In Section III, detailed EKFNet backpropagation is discussed. In Section IV, the dataset, training parameters, and results are presented. The conclusion and future work are provided in Section V.

II. EKF AND LOSS FUNCTIONS

In this paper, we propose a framework for the EKF to learn the system noise covariance parameters, which is trained by the BPTT and gradient descent methods [39], [40], [41]. In this section, the basic EKFNet forward operation is discussed in the beginning. Then the loss functions used for training the EKFNet are presented.

A. EKFNet Forward Operation

We begin with describing the background of EKF filtering and different modules for the EKFNet, and defining the basic notations and terminology. The forward operation of the EKFNet is the same as the EKF [1], [2], and separated into three different modules: the state prediction, measurement prediction, and state update modules, as illustrated in Fig. 1. The state prediction module is used for predicting the state using the motion model. The measurement prediction module is applied to make a prediction for the next measurement and calculate its residual and covariance matrix. The update module updates the system state by combining the predicted state with the measurement residual. The output from the update module is the updated state, also called the posterior, which along with its corresponding covariance matrix, will be used as the prior in the next time step.

1) *State Transition and Measurement Models*: In the EKF the state transition and observation models do not have to be linear functions. At time k , let us denote the system state as $\mathbf{x}_k$, and the measurement as $\mathbf{z}_k$. The motion and measurement models are provided as follows:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}) + \mathbf{\Gamma}_k \mathbf{v}_k(\mathbf{x}_{k-1}) \quad (1a)$$

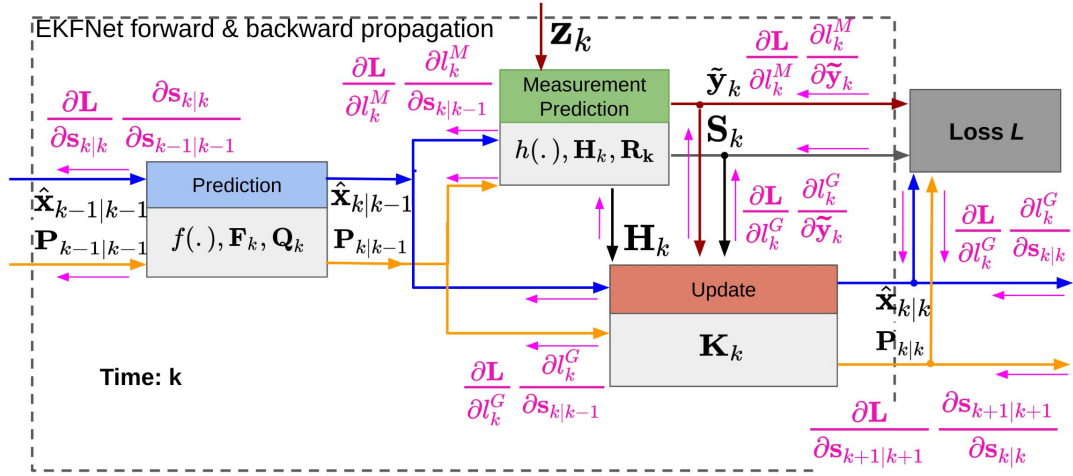
$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{w}_k \quad (1b)$$

where $f(\cdot)$ and $h(\cdot)$ are nonlinear functions, whose corresponding Jacobian matrices are provided below and will be used in the EKF to update the covariance matrices.

$$\mathbf{F}_k = \left. \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_{k-1|k-1}} \quad (2a)$$

$$\mathbf{H}_k = \left. \frac{\partial h(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_k|k-1} \quad (2b)$$

$\mathbf{v}_k(\mathbf{x}_{k-1})$ and $\mathbf{w}_k$ are the process and measurement noises, which are assumed to be independent zero-mean multivariate

Fig. 1. EKFNet diagram. $\mathbf{s} = \{\hat{\mathbf{x}}, \mathbf{P}\}$.

Gaussian random variables with covariance matrices $\mathbf{Q}_k(\mathbf{x}_{k-1})$ and $\mathbf{R}_k$, respectively. Note that in general the process noise and the system state have different dimensions, and $\mathbf{\Gamma}_k$ is the process noise gain matrix. The goal is to learn the parameters in $\mathbf{Q}_k(\mathbf{x}_{k-1})$ and $\mathbf{R}_k$ with real data.

2) *Prediction Module*: The prediction module in Fig. 1 is used for state prediction. Notation $\hat{\mathbf{x}}_{k|m}$ represents the estimate of $\mathbf{x}$ at time k given the measurements up to time m . The state prediction equations are:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}) \quad (3a)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{\Gamma}_k \mathbf{Q}_k(\hat{\mathbf{x}}_{k-1|k-1}) \mathbf{\Gamma}_k^T \quad (3b)$$

$\mathbf{Q}_k$ is the time and state dependent process noise covariance matrix, which is controlled by a set of constant parameters, denoted as β . Such an example will be provided later in Section IV. For the prediction module, the input is the posterior from the previous time step, and the output is the predicted state and its covariance according to the state transition model in (1a).

3) *Measurement Prediction Module*: For the measurement prediction module in Fig. 1, the task is to evaluate the measurement residual $\tilde{\mathbf{y}}_k$ and its covariance $\mathbf{S}_k$:

$$\tilde{\mathbf{y}}_k = \mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1}) \quad (4a)$$

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k \quad (4b)$$

where $\tilde{\mathbf{y}}_k$ is the measurement residual and $\mathbf{S}_k$ is the residual covariance matrix, which will be used later in Section II-B to calculate the measurement loss function. $\mathbf{R}_k$ is the measurement noise covariance matrix, which is controlled by a set of constant parameters, denoted as η .

4) *Update Module*: Based on the previous two steps, the update module in Fig. 1 updates the posterior state estimate. First, the Kalman gain $\mathbf{K}_k$ is calculated as follows:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{S}_k^{-1} \quad (5)$$

Once the Kalman gain is obtained, the state estimate is updated with the measurement residual and its corresponding covariance matrix is also calculated as follows:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \tilde{\mathbf{y}}_k \quad (6a)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} \quad (6b)$$

Finally, the updated state estimate and its covariance will be passed to the next time step as the prior. The same operations will be repeated recursively over time until the last measurement is incorporated by the EKF to update the state estimate.

B. Loss Functions

In order to achieve a better EKF performance, we present four optimization criteria, which can be used to optimize the noise covariances either with or without ground truth system states.

1) *Measurement Residual*: A loss function can be constructed using the l_2 norm of the measurement residual. The measurement residual $\tilde{\mathbf{y}}_k$ is the output of the measurement prediction module, defined in Section II-A3, and the loss function based on measurement residuals can be calculated from time 1 to T as follows:

$$L_{Res}^M = \sum_{k=1}^T \|\tilde{\mathbf{y}}_k\|_2^2 \quad (7)$$

2) *Mean Squared Error (MSE) of State Estimate*: The MSE of the updated state is calculated between the ground truth $\mathbf{x}_k^G$ and a projection from $\hat{\mathbf{x}}_{k|k}$, and the corresponding loss function is:

$$L_{MSE}^G = \sum_{k=1}^T \|\mathbf{x}_k^G - g_k(\hat{\mathbf{x}}_{k|k})\|_2^2 \quad (8)$$

where $g_k(\cdot)$ is a projection function that transforms the full state to the sub-space of the ground truth. Note that high-end sensor data or human-annotated references are often used as ground truth. Hence, the ground truth is expensive to obtain in some applications.

3) *Measurement Residual Log-Likelihood*: The loss function defined in (7) is based only on the measurement residual, but not on its covariance matrix. At each time, the measurement residual covariance matrix $\mathbf{S}_k$ is also calculated by the EKF. So hereby maximizing the likelihood, we consider not only the residual but also its uncertainty. By taking the logarithm of the likelihood, we can deal with additions instead of multiplications. The negative log-likelihood of the measurements is provided in (9).

$$\begin{aligned} L_L^M &= -\log \left(\prod_{k=1}^T p(\mathbf{z}_k | \mathbf{z}_{0:k-1}) \right) \\ &= -\sum_{k=1}^T \log p(\mathbf{z}_k | \mathbf{z}_{0:k-1}) \\ &= \frac{1}{2} \sum_{k=1}^T (\log(|\mathbf{S}_k|) + \tilde{\mathbf{y}}_k^T \mathbf{S}_k^{-1} \tilde{\mathbf{y}}_k + \text{const}) \end{aligned} \quad (9)$$

where the last step is due to the Gaussian approximation used for the measurement residual in the EKF. Compared with (7), in (9) the measurement residual covariance $\mathbf{S}_k$ is incorporated in the loss function.

4) *Posterior Log-Likelihood*: The loss function based on the MSE between the estimated state and the ground truth, as defined in (8), involves only the estimation error itself, but ignores its covariance. By considering the posterior's uncertainty which is the posterior covariance $\mathbf{P}_{k|k}$, the negative log-likelihood of the ground truth is defined in (10).

$$\begin{aligned} L_L^G &= -\log \left(\prod_{k=1}^T p(\mathbf{x}_k^G | \mathbf{z}_{0:k}) \right) \\ &= -\sum_{k=1}^T \log p(\mathbf{x}_k^G | \mathbf{z}_{0:k}) \\ &= \frac{1}{2} \sum_{k=1}^T \left(\log(|\mathbf{P}_{k|k}^G|) + \tilde{\mathbf{x}}_k^T (\mathbf{P}_{k|k}^G)^{-1} \tilde{\mathbf{x}}_k + \text{const} \right) \end{aligned} \quad (10)$$

where

$$\tilde{\mathbf{x}}_k = \mathbf{x}_k^G - g_k(\hat{\mathbf{x}}_{k|k}) \quad (11)$$

and $\mathbf{P}_{k|k}^G$ is the covariance matrix of $\mathbf{x}_k^G$, and can be constructed using the corresponding sub-matrix from $\mathbf{P}_{k|k}$. Again, the last step of (10) is derived because of the Gaussian assumption used for $\tilde{\mathbf{x}}_k$ in the EKF.

Note that the measurement negative log-likelihood in (9) can be rigorously derived. However, the ground-truth state negative log-likelihood in (10) cannot be systematically derived. Rather, it is constructed as a cost function for the convenience of training the EKFNet using ground truth data.

The constant terms in (9) and (10) can be ignored during backpropagation because these terms have zero gradients with respect to the optimization variables. The physical meaning of (9) and (10) is that the EKF should be tuned to maximize the likelihood corresponding to the measurements or to the updated state estimates. Because in (7) and (9) only the measurement

data are needed, the optimization criterion based on the measurement residual or the measurement likelihood can be applied without the ground truth data.

5) *Loss Functions With Regularization*: In many applications, it is often desirable not only to minimize the loss functions defined earlier in this section but also to encourage an optimization solution with other properties, such as its closeness to a manually optimized solution. Hence, we propose two regularized loss functions, based on either the measurement residuals/MSE errors or the negative log-likelihoods:

$$L_{MSE} = aL_{MSE}^G + bL_{MSE}^M + \lambda_1 r_1(\mathbf{Q}) + \lambda_2 r_2(\mathbf{R}) \quad (12)$$

$$L_{Like} = aL_L^G + bL_L^M + \lambda_1 r_1(\mathbf{Q}) + \lambda_2 r_2(\mathbf{R}) \quad (13)$$

Each loss function consists of two parts: the first two terms are loss functions and the last two terms are regularization functions ($r(\cdot)$ functions). The loss functions encourage the filter output to minimize the residual/MSE error or maximize the likelihood. The regularization functions are problem-dependent but have to be differentiable. For example, it can encourage the covariance matrices to be close to the manually optimized results [11]. a , b , λ_1 , and λ_2 are the weights for loss functions or regularization functions and they are hyper parameters. In applications without ground truth, the first term can be ignored, i.e. $a = 0$.

To simplify the notations in the next section (Section III), (12) and (13) can be unified in a single equation as follows:

$$\begin{aligned} L &= aL^G + bL^M + \lambda_1 r_1(\mathbf{Q}) + \lambda_2 r_2(\mathbf{R}) \\ &= a \sum_{k=1}^T l_k^G + b \sum_{k=1}^T l_k^M + \lambda_1 r_1(\mathbf{Q}) + \lambda_2 r_2(\mathbf{R}) \end{aligned} \quad (14)$$

where the definitions of l_k^G and l_k^M depend on which loss function has been adopted. When L_{MSE} is adopted as the loss function L , based on (7) and (8)

$$l_k^M \triangleq \|\tilde{\mathbf{y}}_k\|_2^2 \quad (15)$$

and

$$l_k^G \triangleq \|\mathbf{x}_k^G - g_k(\hat{\mathbf{x}}_{k|k})\|_2^2 \quad (16)$$

On the other hand, if L_{Like} is used as the loss function, then from (9) and (10)

$$l_k^M \triangleq \frac{1}{2} (\log(|\mathbf{S}_k|) + \tilde{\mathbf{y}}_k^T \mathbf{S}_k^{-1} \tilde{\mathbf{y}}_k) \quad (17)$$

and

$$l_k^G \triangleq \frac{1}{2} \left(\log(|\mathbf{P}_{k|k}^G|) + \tilde{\mathbf{x}}_k^T (\mathbf{P}_{k|k}^G)^{-1} \tilde{\mathbf{x}}_k \right) \quad (18)$$

III. BACK PROPAGATION THROUGH TIME

BPTT is a gradient-based method for training a directed computation graph based on time sequence, which is widely used to train the RNN [37], [40], [41]. The EKF can be deemed as a directed graph, which is built by internal states and measurement data of variable lengths. The training data for the EKF are a sequence of T input-output pairs $\{\mathbf{z}_k, (\hat{\mathbf{x}}_{k|k}, \mathbf{P}_{k|k}, \tilde{\mathbf{y}}_k, \mathbf{S}_k)\}_{k=1}^T$. For the purpose of optimizing the EKF performance, BPTT

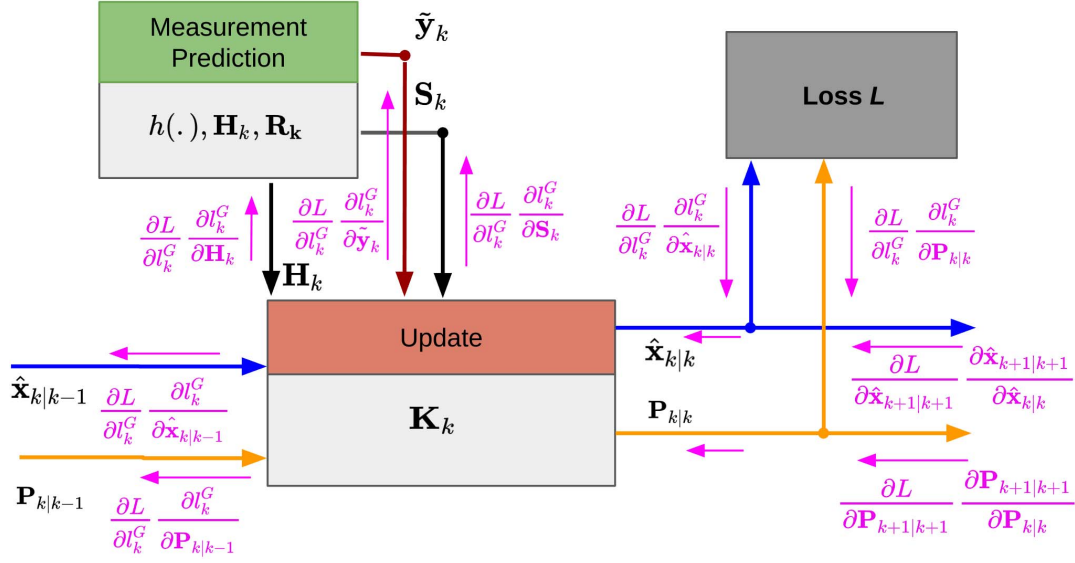


Fig. 2. Backpropagation in the update module.

begins with unfolding the EKF over time, and each node uses the same parameters (β and η) to construct $\mathbf{Q}$ and $\mathbf{R}$, as discussed later in Section IV-B. Then the backpropagation algorithm is used to find the gradients of the loss function with respect to (w.r.t.) the parameters in $\mathbf{Q}$ and $\mathbf{R}$. Finally, the gradient descent optimization algorithm will be used to update the parameters based on the gradients [39], [40], [41].

The backpropagation flow is shown in Fig. 1. The pink equations and left arrows indicate the backward gradient flow. At each step k , the gradients are derived from: (1) the loss function l_k^M , (2) loss function l_k^G , and (3) gradients from time step $k+1$.

In this section, a detailed backpropagation procedure for each module and each loss function will be discussed. The gradients, which can be used to optimize the process and measurement noise covariance parameters, are constructed at the end of backpropagation. Because backpropagation is based on the chain rule for the gradient [41], here we only list the gradient results w.r.t. the intermediate parameters for brevity.

A. Back Propagation in Update Module

The forward operation for the update module has been discussed in Section II-A4. In this subsection, we focus on backpropagation in the update module. The gradients input to the update module are those of the loss function w.r.t. the posterior states $\hat{\mathbf{x}}_{k|k}$ and its covariance $\mathbf{P}_{k|k}$. As shown in Fig. 2, the gradient can be constructed by the following equations:

$$\frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k}} = \frac{\partial L}{\partial \hat{\mathbf{x}}_{k+1|k+1}} \frac{\partial \hat{\mathbf{x}}_{k+1|k+1}}{\partial \hat{\mathbf{x}}_{k|k}} + \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \hat{\mathbf{x}}_{k|k}} \quad (19a)$$

$$\frac{\partial L}{\partial \mathbf{P}_{k|k}} = \frac{\partial L}{\partial \mathbf{P}_{k+1|k+1}} \frac{\partial \mathbf{P}_{k+1|k+1}}{\partial \mathbf{P}_{k|k}} + \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{P}_{k|k}} \quad (19b)$$

In (19a) and (19b), the first term on the right hand side is the gradient flow arriving from time step $k+1$, and the second term is the gradient of the state estimation error for the current step k denoted as l_k^G . As shown in Fig. 2, the gradient flow for the update module can be summarized in (20), where U_B stands for the backward operation for the update module for simplicity.

As shown in Fig. 2 and (20), there are five terms output from the update module. The first two terms are the gradients w.r.t. the predicted state $\hat{\mathbf{x}}_{k|k-1}$ and its covariance $\mathbf{P}_{k|k-1}$, which will flow back to the prediction module. The rest three terms are the gradients of the loss function w.r.t. the residual $\tilde{\mathbf{y}}_k$, its covariance $\mathbf{S}_k$, and the measurement matrix $\mathbf{H}_k$, which will flow back to the measurement prediction module.

Based on the chain rule, the gradients can be calculated analytically. Backpropagation for the update module's outputs to the prediction module are provided below:

$$\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \hat{\mathbf{x}}_{k|k-1}} = \frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k}} \quad (21)$$

$$\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{P}_{k|k-1}} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T \frac{\partial L}{\partial \mathbf{P}_{k|k}} + \frac{\partial L}{\partial \mathbf{K}_k} \mathbf{S}_k^{-1} \mathbf{H}_k \quad (22)$$

where:

$$\frac{\partial L}{\partial \mathbf{K}_k} = \frac{\partial L}{\partial \tilde{\mathbf{y}}_k} \tilde{\mathbf{y}}_k^T - \frac{\partial L}{\partial \mathbf{P}_{k|k}} \mathbf{P}_{k|k-1} \mathbf{H}_k^T \quad (23)$$

$$\left(\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \hat{\mathbf{x}}_{k|k-1}}, \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{P}_{k|k-1}}, \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \tilde{\mathbf{y}}_k}, \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{S}_k}, \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{H}_k} \right) = U_B \left(\frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k}}, \frac{\partial L}{\partial \mathbf{P}_{k|k}} \right) \quad (20)$$

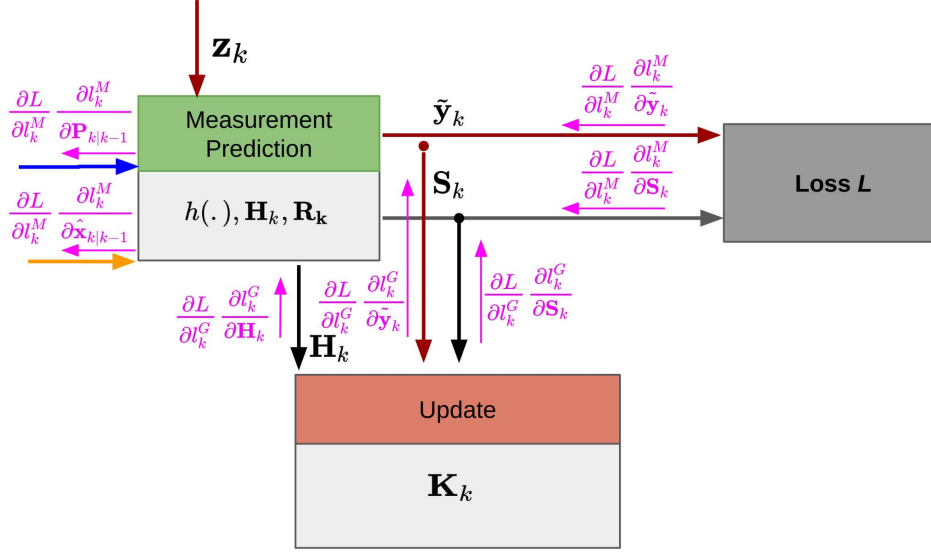


Fig. 3. Back propagation in the measurement prediction module.

$\frac{\partial L}{\partial l_k^G}$ is the derivative of the loss function L w.r.t. the state estimation error l_k^G at time step k . The detailed derivations of (21), (22), and (23) are provided in Appendix A.

Next, we consider the back propagation from the update module to the measurement prediction module. In this process, as characterized in (20), first the derivatives of L w.r.t. $\mathbf{P}_{k|k}$ and $\hat{\mathbf{x}}_{k|k}$ ($\frac{\partial L}{\partial \mathbf{P}_{k|k}}$ and $\frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k}}$) flow into the update module. They are then used to calculate the derivatives of L w.r.t. $\tilde{\mathbf{y}}_k$, $\mathbf{H}_k$, and $\mathbf{S}_k$, which will then flow from the update module back to the measurement prediction module, and are provided as follows:

$$\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \tilde{\mathbf{y}}_k} = \mathbf{K}_k^T \frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k}} \quad (24)$$

$$\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{H}_k} = -\mathbf{K}_k^T \frac{\partial L}{\partial \mathbf{P}_{k|k}} \mathbf{P}_{k|k-1} + \left(\mathbf{P}_{k|k-1} \frac{\partial L}{\partial \mathbf{K}_k} \mathbf{S}_k^{-1} \right)^T \quad (25)$$

$$\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{S}_k} = -\mathbf{S}_k^{-1} \mathbf{H}_k \mathbf{P}_{k|k-1} \frac{\partial L}{\partial \mathbf{K}_k} \mathbf{S}_k^{-1} \quad (26)$$

The derivation of (24)-(26) is provided in Appendix B. The partial derivatives in the following sections can be proved in similar manners, whose detailed derivations will be skipped for brevity of the paper.

B. Back Propagation in Measurement Prediction Module

The forward operation for the measurement prediction module has been discussed in Section II-A3. The gradients input to the measurement prediction module are the gradients of loss function w.r.t. the residual $\tilde{\mathbf{y}}_k$ and its covariance $\mathbf{S}_k$, and the gradient outputs from the update module which have been discussed in Section III-A. As shown in Fig. 3, the gradient can be constructed by three terms for the residual $\tilde{\mathbf{y}}_k$, $\mathbf{S}_k$, and

$\mathbf{H}_k$, which are given in (24), (25), and (26). The inputs can be constructed according to the equation below.

$$\frac{\partial L}{\partial \tilde{\mathbf{y}}_k} = \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \tilde{\mathbf{y}}_k} + \frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \tilde{\mathbf{y}}_k} \quad (27a)$$

$$\frac{\partial L}{\partial \mathbf{S}_k} = \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{S}_k} + \frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \mathbf{S}_k} \quad (27b)$$

where the first terms on right hand side of (27a) and (27b) are the outputs from the update module. The second terms on the right hand side of (27a) and (27b) are the derivatives w.r.t. the residual error at time step k and its covariance. l_k^M represents the loss function due to the measurement prediction for the current step.

For the measurement prediction module, we can simply summarize the inputs and outputs as:

$$\begin{aligned} & \left(\frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \mathbf{P}_{k|k-1}}, \frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \hat{\mathbf{x}}_{k|k-1}} \right) \\ &= M_B \left(\frac{\partial L}{\partial \tilde{\mathbf{y}}_k}, \frac{\partial L}{\partial \mathbf{S}_k}, \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{H}_k} \right) \end{aligned} \quad (28)$$

where M_B is the backward operation for the measurement prediction module.

The gradients that propagate from the measurement prediction module to the prediction module are shown below:

$$\frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \hat{\mathbf{x}}_{k|k-1}} = -\mathbf{H}_k^T \frac{\partial L}{\partial \tilde{\mathbf{y}}_k} + \mathbf{d}_k \quad (29)$$

$$\frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \mathbf{P}_{k|k-1}} = \mathbf{H}_k^T \frac{\partial L}{\partial \mathbf{S}_k} \mathbf{H}_k \quad (30)$$

Note that $\mathbf{d}_k$ is a $n_x \times 1$ vector, where n_x is the dimension of the system state $\mathbf{x}_k$, its l -th element is

$$\mathbf{d}_k(l) = \sum_i \sum_j \left(\mathbf{B}_k \odot \frac{\partial \mathbf{H}_k}{\partial \hat{\mathbf{x}}_{k|k-1}(l)} \right)_{i,j} \quad (31)$$

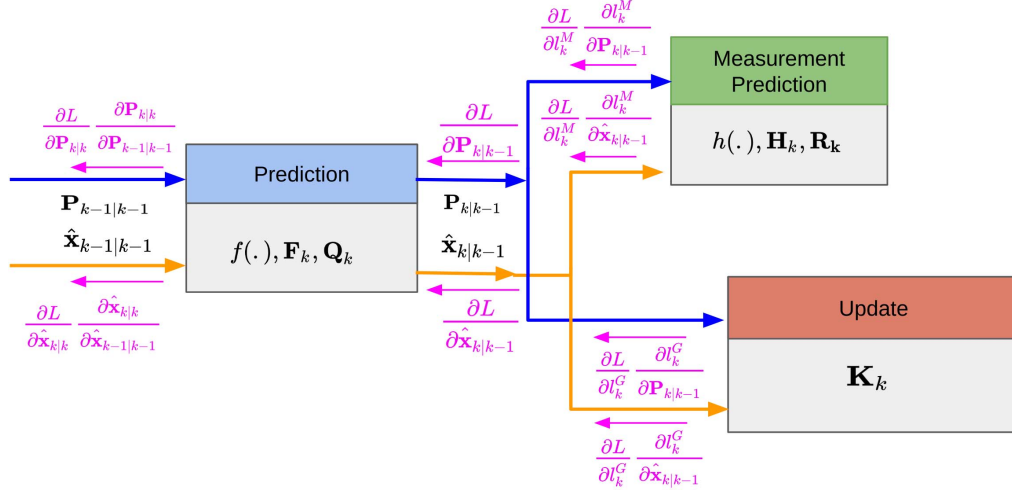


Fig. 4. Back propagation in the prediction module.

for $l = 1, \dots, n_x$, $\odot$ denotes the Hadamard (element-wise) matrix product, and

$$\mathbf{B}_k = \left(\mathbf{P}_{k|k-1}^T \mathbf{H}_k^T \frac{\partial L}{\partial \mathbf{S}_k} \right)^T + \frac{\partial L}{\partial \mathbf{S}_k} \mathbf{H}_k \mathbf{P}_{k|k-1}^T + \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{H}_k} \quad (32)$$

is the derivative $\frac{\partial L}{\partial \mathbf{H}_k}$. Since $\mathbf{H}_k$ is also a function of the predicted state $\hat{\mathbf{x}}_{k|k-1}$, the derivative $\mathbf{H}_k$ w.r.t. each element of the predicted state $\hat{\mathbf{x}}_{k|k-1}$ also needs to be calculated, which results in the second term on the right hand side of (29).

Because the measurement noise covariance is involved in the measurement prediction module, the gradients w.r.t. the measurement noise covariance $\mathbf{R}_k$ can be calculated during backpropagation:

$$\frac{\partial L}{\partial \mathbf{R}_k} = \frac{\partial L}{\partial \mathbf{S}_k} \quad (33)$$

C. Back Propagation in Prediction Module

The forward operation in the prediction module has been discussed in Section II-A2. As shown in Fig. 4, the gradients propagating back to the prediction module are the gradients of the loss function w.r.t. the predicted state $\hat{\mathbf{x}}_{k|k-1}$ and its covariance $\mathbf{P}_{k|k-1}$, which are constructed by combining outputs from the measurement prediction module and update module, according to the equation below.

$$\frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k-1}} = \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \hat{\mathbf{x}}_{k|k-1}} + \frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \hat{\mathbf{x}}_{k|k-1}} \quad (34a)$$

$$\frac{\partial L}{\partial \mathbf{P}_{k|k-1}} = \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{P}_{k|k-1}} + \frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \mathbf{P}_{k|k-1}} \quad (34b)$$

By combining the outputs from (20) and (28), the backward propagation for the prediction module can be summarized as

in (35), where P_B is the backward operation for the prediction module. The output gradients will be passed to the update module at time step $k-1$.

$$\frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k}} \frac{\partial \hat{\mathbf{x}}_{k|k}}{\partial \hat{\mathbf{x}}_{k-1|k-1}} = \mathbf{F}_k^T \frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k-1}} + \mathbf{e}_k \quad (36)$$

$$\frac{\partial L}{\partial \mathbf{P}_{k|k}} \frac{\partial \mathbf{P}_{k|k}}{\partial \mathbf{P}_{k-1|k-1}} = \mathbf{F}_k^T \frac{\partial L}{\partial \mathbf{P}_{k|k-1}} \mathbf{F}_k \quad (37)$$

Note that $\mathbf{e}_k$ is a $n_x \times 1$ vector, its l -th element is

$$\mathbf{e}_k(l) = \sum_i \sum_j \left(\mathbf{C}_k \odot \frac{\partial \mathbf{F}_k}{\partial \hat{\mathbf{x}}_{k-1|k-1}(l)} \right)_{i,j} + \sum_m \sum_n \left(\mathbf{D}_k \odot \frac{\partial \mathbf{Q}_k}{\partial \hat{\mathbf{x}}_{k-1|k-1}(l)} \right)_{m,n} \quad (38)$$

for $l = 1, \dots, n_x$, where

$$\mathbf{C}_k = \frac{\partial L}{\partial \mathbf{P}_{k|k-1}} \mathbf{F}_k \mathbf{P}_{k-1|k-1}^T + \left(\mathbf{P}_{k-1|k-1}^T \mathbf{F}_k^T \frac{\partial L}{\partial \mathbf{P}_{k|k-1}} \right)^T \quad (39)$$

is the gradient of the loss function w.r.t. $\mathbf{F}_k$, and

$$\mathbf{D}_k = \frac{\partial L}{\partial \mathbf{Q}_k} = \mathbf{F}_k^T \frac{\partial L}{\partial \mathbf{P}_{k|k-1}} \mathbf{F}_k \quad (40)$$

is the gradient of the loss function w.r.t. $\mathbf{Q}_k$, which is derived based on (3b), and following a similar procedure in Appendix B-B. Because $\mathbf{F}_k$ and $\mathbf{Q}_k$ are functions of $\hat{\mathbf{x}}_{k-1|k-1}$, their gradients w.r.t. $\hat{\mathbf{x}}_{k-1|k-1}$ also need to be calculated during backpropagation, which is the second term on the right-hand side of (36). The first term in (36) is derived based on the forward operation (3a), where the gradient of the dynamic model is the Jacobian matrix $\mathbf{F}_k$.

$$\left(\frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k}} \frac{\partial \hat{\mathbf{x}}_{k|k}}{\partial \hat{\mathbf{x}}_{k-1|k-1}}, \frac{\partial L}{\partial \mathbf{P}_{k|k}} \frac{\partial \mathbf{P}_{k|k}}{\partial \mathbf{P}_{k-1|k-1}} \right) = P_B \left(\frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k-1}}, \frac{\partial L}{\partial \mathbf{P}_{k|k-1}} \right) \quad (35)$$

Note that in the EKFNNet, to find the optimal process noise covariance parameters, the gradient of the loss function w.r.t. $\mathbf{Q}_k$ is needed, which has been provided in (40).

D. Back Propagation of Loss Function

The gradients for the loss function are different when different optimization criteria are chosen. Among the four different loss functions, some of them only have gradients w.r.t. the state, and some of them have gradients w.r.t. both the state and its covariance.

1) *Back Propagation of Residual Error*: The measurement residual is calculated without using the ground truth data. The forward operation of this part is shown in (7). If the loss function only considers the measurement residual, the gradient flowing back to each time steps is listed below:

$$\frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \tilde{\mathbf{y}}_k} = 2b\tilde{\mathbf{y}}_k \quad (41)$$

The gradient of the loss function w.r.t. $\mathbf{S}_k$ is zero in this case.

2) *Back Propagation of Negative Log Residual Likelihood*: In this section, the back propagation operation for the negative log residual likelihood function is derived, by considering not only the measurement residual $\tilde{\mathbf{y}}_k$, but also its covariance $\mathbf{S}_k$. The loss function used here is defined in (9). The gradients calculated from backward propagation flow back to the measurement prediction module. The gradients of the loss function w.r.t. $\mathbf{S}_k$ and $\tilde{\mathbf{y}}_k$ are shown below:

$$\frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \tilde{\mathbf{y}}_k} = b\mathbf{S}_k^{-1}\tilde{\mathbf{y}}_k \quad (42)$$

$$\frac{\partial L}{\partial l_k^M} \frac{\partial l_k^M}{\partial \mathbf{S}_k} = \frac{b}{2} [\mathbf{S}_k^{-1} - \mathbf{S}_k^{-1}\tilde{\mathbf{y}}_k\tilde{\mathbf{y}}_k^T\mathbf{S}_k^{-1}] \quad (43)$$

3) *Back propagation of MSE of State Estimate*: To find the state estimation error, the ground truth data are required for the system state. The gradient from this section flow back to the update module and the loss function w.r.t. $\mathbf{P}_{k|k}$ is 0 in this case. The gradient flowing back to each time step can be calculated as follows:

$$\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \tilde{\mathbf{x}}_{k|k}} = 2a\tilde{\mathbf{x}}_k \quad (44)$$

where $\tilde{\mathbf{x}}_k$ is defined in (11), which is the error between the state and ground truth. Note that in this paper, for simplicity and without loss of generality, we assume that $\mathbf{x}_k^G = \mathbf{x}_k$.

4) *Back Propagation of Posterior State Log-Likelihood*: To evaluate the posterior state likelihood, one needs the state ground truth data. The back propagation chain rule for the posterior state log-likelihood has been derived and given below:

$$\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \tilde{\mathbf{x}}_{k|k}} = a\mathbf{P}_{k|k}^{-1}\tilde{\mathbf{x}}_k \quad (45)$$

$$\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{P}_{k|k}} = \frac{a}{2} [\mathbf{P}_{k|k}^{-1} - \mathbf{P}_{k|k}^{-1}\tilde{\mathbf{x}}_k\tilde{\mathbf{x}}_k^T\mathbf{P}_{k|k}^{-1}] \quad (46)$$

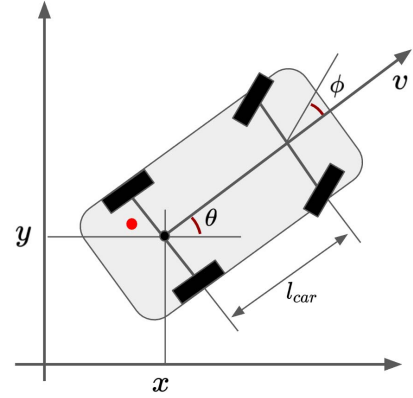


Fig. 5. Vehicle motion model, l_{car} is 2.71m.

E. Derivative of Loss w.r.t. the Noise Covariance Parameters

For most of the cases, the training dataset has multiple tracks, with possibly different track duration T^i s. The derivatives w.r.t. the process and measurement noise covariance parameters β and η are calculated at each step as in (47a) and (47b) respectively for each track i and they are accumulated over time steps and different tracks by using (33) and (40):

$$\frac{\partial L}{\partial \beta} = \sum_{i=1}^N \sum_{k=1}^{T^i} \frac{\partial L^i}{\partial \mathbf{Q}_k^i} \frac{\partial \mathbf{Q}_k^i}{\partial \beta} \quad (47a)$$

$$\frac{\partial L}{\partial \eta} = \sum_{i=1}^N \sum_{k=1}^{T^i} \frac{\partial L^i}{\partial \mathbf{R}_k^i} \frac{\partial \mathbf{R}_k^i}{\partial \eta} \quad (47b)$$

where i represents the track identity, and N is the total number of the tracks. T^i is the total number of steps for the i th track. (47a) and (47b) can be derived and presented in a manner similar to $\mathbf{d}_k$ and $\mathbf{e}_k$ as in Sections III-B and III-C, which is skipped here for brevity.

Many optimization methods have been constructed based on gradients. The variables can be iteratively updated towards the optimal direction by gradient descent [40], [41].

IV. EXPERIMENTS

Here we provide an example where a vehicle is tracked in a 2D space, using data from KITTI dataset [42]. The GPS measurements include longitude, latitude, and orientation. The ground truth states are generated by using extended Kalman smoother [1] at the original sampling rate of 100 Hz, and the measurements are generated by down-sampling the original data to 2 Hz and adding small Gaussian noise for testing the algorithm.

A. Motion and Measurement Models

1) *Motion Model*: In this work, we use a model that closely represents the vehicle motion by the application of nonholonomic constraints as proposed in [43], [44], [45], [46]. As shown in Fig. 5, the states are:

$$\mathbf{x}_k = [x_k, y_k, \theta_k, v_k, \phi_k]^T \quad (48)$$

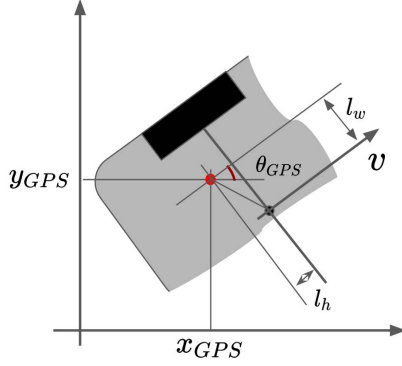


Fig. 6. Vehicle measurement model, l_w is 0.32m and l_h is 0.05m.

where x_k and y_k are the rear center coordinates, θ_k is the global vehicle heading, v_k is the linear velocity, and ϕ_k is the steering angle. We assume that the vehicle is a front-wheel drive, and the motion model $f(\cdot)$ governs how the vehicle state transitions according to current states and time [43], [46]:

$$x_{k+1} = \cos(\theta_k) \times \cos(\phi_k) \times v_k \times \Delta t + x_k \quad (49a)$$

$$y_{k+1} = \sin(\theta_k) \times \cos(\phi_k) \times v_k \times \Delta t + y_k \quad (49b)$$

$$\theta_{k+1} = \sin(\phi_k) \times v_k \times \Delta t / l_{car} + \theta_k \quad (49c)$$

$$v_{k+1} = v_k \quad (49d)$$

$$\phi_{k+1} = \phi_k \quad (49e)$$

The Jacobian matrix for motion model can be found in (50), which is constructed at each time step.

In the motion model, we assume that the steering angle ϕ_k and velocity v_k remain constant from one time step to the next. This model is usually called Constant Turn Rate and Velocity (CTRV) model [47]. The main feature of the kinematic model of a car like vehicle is the presence of two nonholonomic constraints in (49) refraining the front/rear wheels from moving slide ways.

2) *Measurement Model*: By using the GPS data, the measurements are the center position x_k^{GPS} , y_k^{GPS} , and the heading angle θ_k^{GPS} of the vehicle:

$$\mathbf{z}_k = [x_k^{GPS}, y_k^{GPS}, \theta_k^{GPS}]^T \quad (51)$$

The measurement model $h(\cdot)$ is provided in (52), in which $l_{off} = 0.323$ is the distance between the GPS receiver and the vehicle rear center, and $\alpha = 0.154$ is the angle between x-axis

and the line of sight between the GPS receiver and the rear center, as illustrated in Fig. 6.

$$x_k = x_k^{GPS} - l_{off} \cos(\hat{\theta}_{k|k-1} + \alpha + 0.5\pi) \quad (52a)$$

$$y_k = y_k^{GPS} - l_{off} \sin(\hat{\theta}_{k|k-1} + \alpha + 0.5\pi) \quad (52b)$$

$$\theta = \theta_k^{GPS} \quad (52c)$$

The Jacobian matrix for the measurement model, $\mathbf{H}_k$, is derived and provided as follows:

$$\mathbf{H}_k = \begin{bmatrix} 1 & 0 & 0 & 0 & l_{off} \sin(\hat{\theta}_{k|k-1} + \alpha + 0.5\pi) \\ 0 & 1 & 0 & 0 & -l_{off} \cos(\hat{\theta}_{k|k-1} + \alpha + 0.5\pi) \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \quad (53)$$

B. Process and Measurement Noise Covariance Matrices

For simplicity, the model assumes the constant velocity and constant steering rate, which is not realistic in real tracking applications. The state-dependent process noise covariance matrix is given in (54), where σ_A is the maximum acceleration and σ_{SR} is the maximum steering rate due to physical limitations. The definition of $\mathbf{G}(\cdot)$ is provided in (55), which is the dynamic model for acceleration and steering rate [46]. It is clear that the parameter vector β , that controls $\mathbf{Q}_k$, consists of the seven σ terms in (54).

The measurement noise covariance is provided in (56).

$$\mathbf{R}_k = \text{diag}(\sigma_{GPS_x}^2, \sigma_{GPS_y}^2, \sigma_{GPS_\theta}^2) \quad (56)$$

Hence, the parameter vector η , that controls $\mathbf{R}_k$, consists of the three σ terms in (56). Note that even though the process noise covariance matrix $\mathbf{Q}_k$ is state dependent, the EKFNet only learns all the constant σ terms in (54) or equivalently β , but not the state dependent $\mathbf{G}_k(\mathbf{x}_k)$ terms. As long as $\mathbf{G}_k(\mathbf{x}_k)$ is differentiable w.r.t. $\mathbf{x}_k$, the EKFNet is able to perform the back propagation properly. In a more general case, where the dependence of the covariance matrix $\mathbf{Q}_k$ on the system state $\mathbf{x}_k$ is not explicitly known to the EKFNet as in (54), one can incorporate a neural network into the EKFNet to learn the relationship between $\mathbf{Q}_k$ and $\mathbf{x}_k$.

C. Experiment Results

The whole dataset is separated to training, validation, and testing sets. The validation set is used during the training stage to prevent overfitting.

$$\mathbf{F}_k = \begin{bmatrix} 1 & 0 & -\sin(\theta_k) \cos(\phi_k) v_k \Delta t & \cos(\theta_k) \cos(\phi_k) \Delta t & -\cos(\theta_k) \sin(\phi_k) v_k \Delta t \\ 0 & 1 & \cos(\theta_k) \cos(\phi_k) v_k \Delta t & \sin(\theta_k) \cos(\phi_k) \Delta t & -\sin(\theta_k) \sin(\phi_k) v_k \Delta t \\ 0 & 0 & 1 & \sin(\phi_k) \Delta t / l_{car} & \cos(\phi_k) v_k \Delta t / l_{car} \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (50)$$

$$\mathbf{Q}_k(\mathbf{x}_k) = \mathbf{G}(\mathbf{x}_k) \text{diag}(\sigma_A^2, \sigma_{SR}^2) \mathbf{G}^T(\mathbf{x}_k) + \text{diag}(\sigma_x^2, \sigma_y^2, \sigma_\theta^2, \sigma_v^2, \sigma_\phi^2) \quad (54)$$

$$\mathbf{G}(\mathbf{x}_k) = \begin{bmatrix} \frac{1}{2}(\Delta t)^2 \cos(\phi_k) & \frac{1}{2}(\Delta t)^2 \sin(\phi_k) & 0 & \Delta t & 0 \\ 0 & 0 & \frac{1}{2}(\Delta t)^2 & 0 & \Delta t \end{bmatrix}^T \quad (55)$$

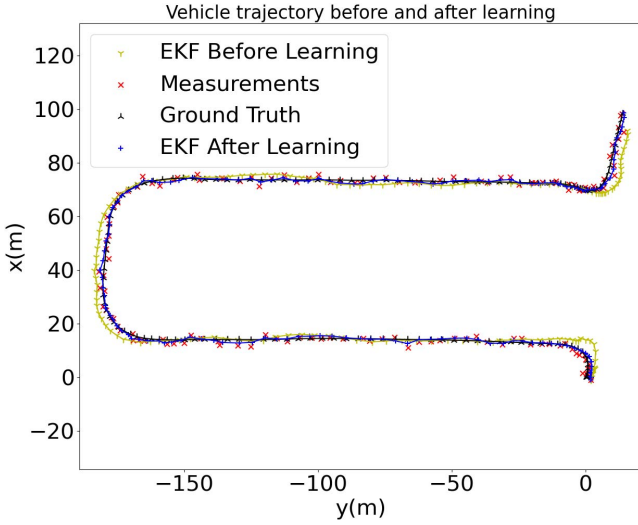


Fig. 7. Estimated vehicle trajectories before and after training.

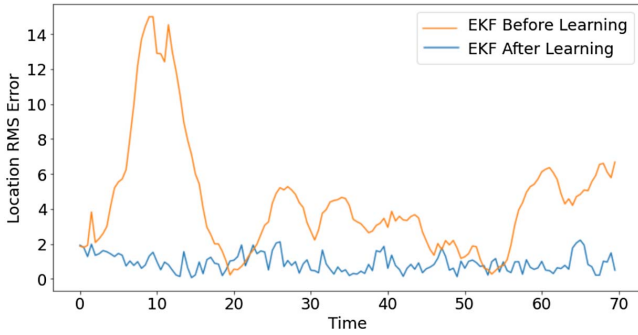
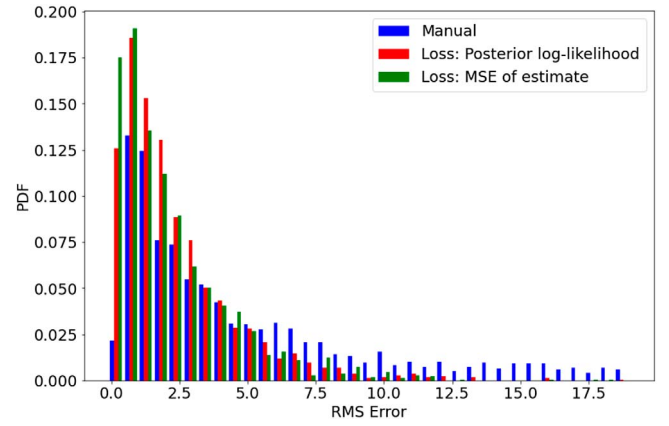


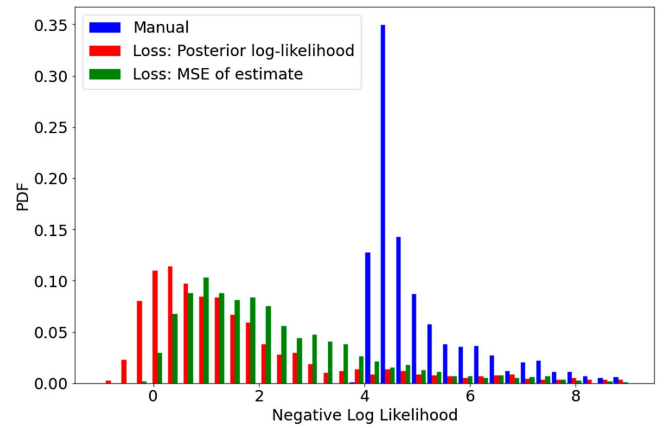
Fig. 8. RMS error for location before and after training.

1) *Vehicle Trajectory Estimates*: In Fig. 7, an example trajectory from the testing set is shown. The yellow trajectory is estimated by the EKF optimized manually, with large errors occurring when the vehicle turns. The blue trajectory is the estimate obtained by the EKFNet, which does not overshoot during the turning period, and is much closer to the ground truth. The loss function used in this case is (12). In Fig. 8, the positional root mean square error (RMSE) is presented by combining MSEs associated with the estimates of the vehicle coordinates x and y . From this figure, it is clear that the trained EKF has a lower estimation error than the original EKF, especially during vehicle turns.

2) *Loss Function Comparison: MSE of State Estimate vs. Posterior Log-Likelihood*: As mentioned earlier, the loss function can be chosen to either include the second order statistic of the measurement residual/state estimation error or not. In this subsection, we compare a loss function based on the MSE of the state estimate error by using (12) and setting $b = 0$, to another one based on the posterior log-likelihood by using (13) and setting $b = 0$. In Fig. 9, the histograms for the positional RMS error and negative posterior log-likelihood are shown. The lower the better for both of the metrics. The manually tuned



(a) Histogram of State Estimation RMS Errors



(b) Histogram of Posterior Negative Log Likelihood

Fig. 9. Loss histograms for EKFs trained using L_{MSE}^G and L_L^G , and a manually tuned EKF.

results are displayed in blue in both of the sub-figures. From Fig. 9(a) the long tail of the histograms for manually tuned EKF can be observed, implying large estimation errors occur with significant probabilities. In Fig. 9(b), for manually tuned EKF, most of negative posterior log-likelihoods are distributed between 4 and 8. The manually tuned covariance matrices are used to initialize the proposed training algorithm.

In the first case, the loss function is based on state estimation MSE L_{MSE}^G , and the corresponding RMS error histogram is shown in Fig. 9 in green. If the loss function is created by considering the posterior state estimate covariance, namely L_L^G , the results are also shown in Fig. 9 in red. Compared with the manually tuned results in blue, the EKFs trained based on the posterior likelihood and the state estimation MSE improve the tracking performance significantly. After training, the long tails of the histograms (PDFs) of the state estimation MSE and the negative log-likelihood have diminished, and these two histograms/distributions have moved to the left with smaller losses on the average.

Further, from this figure, it is clear that a loss function based on likelihoods will lead to a better likelihood results, and a loss function based on MSEs will result in a better MSE

TABLE I
TESTING RESULTS. -LOGL: NEGATIVE LOG-LIKELIHOOD

Loss	RMSE (State)	RMSE (meas)	-LogL (State)	-LogL (meas)
$\mathbf{L}_{MSE}^G + \mathbf{L}_{Res}^M$	1.235	2.434	3.843	5.673
$\mathbf{L}_{MSE}^G$	1.134	2.563	3.745	5.996
$\mathbf{L}_{Res}^M$	1.922	2.367	4.745	5.856
$\mathbf{L}_L^G + \mathbf{L}_L^M$	2.367	3.412	3.562	5.432
$\mathbf{L}_L^G$	1.932	3.758	2.341	5.783
$\mathbf{L}_L^M$	2.745	3.227	3.735	4.735
Hand Tuned	4.246	6.342	8.649	9.354
Joint [12]	4.945	5.354	4.984	5.465

performance. These results are consistent with their loss functions. Further, trained EKFs based on both criteria provide significantly better performance than the manually tuned EKF.

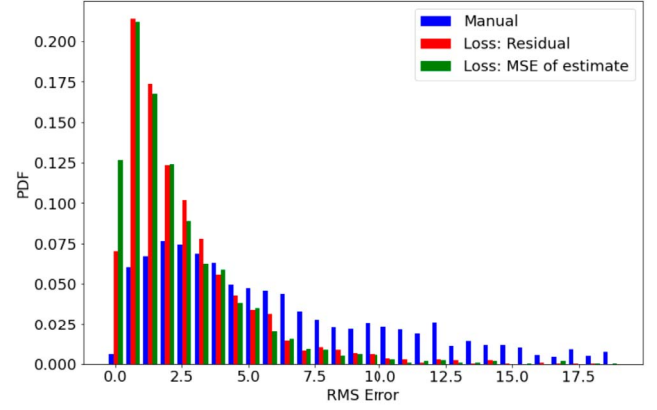
3) *Numerical Results:* In Table I, the testing results are shown. Each row represents a different loss function, and each column stands for a different objective score. The loss functions are constructed as in (12) or (13). It is clear that the best score for each category is achieved by using that category as the loss function alone. If a total loss function is constructed by combining loss functions from two different categories, the scores in both categories are competitive but not the best. Further, the proposed EKFNet method always outperforms the EKF fine-tuned by hand and the Joint (sample covariance) method proposed in [12].

D. Imperfect Motion and Measurement Models

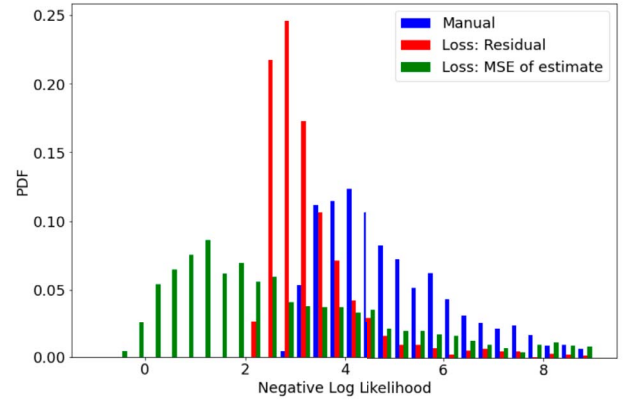
In this section, we will provide the training results using imperfect motion and measurement models. Imperfect models are common for model-based tracking due to the physical limitations. For example, in our case, the vehicle length l_{car} in Fig. 5 and the GPS sensor position in Fig. 6 are not precisely measured, which will cause estimation errors during vehicle tracking. As mentioned before, the process and measurement noise can compensate for these imperfections during tracking. Because the proposed method can be trained with or without the ground truth data, we create two different experiment cases. The first one is with the ground truth and using the state estimation MSE as the loss function L_{MSE}^G . Another one is without the ground truth and using the residual error L_{Res}^M as the loss function. In order to run this experiment, we change l_{car} to 5.0m, $l_{off} = 0.5$ and $\alpha = 1.6$, to make both the motion and measurement model parameters deviate from their true values.

In Fig. 10, the histograms for the state estimation MSE and the posterior negative log-likelihood are provided for two EKFs trained with different loss functions and a manually tuned EKF. The results for the manually tuned EKF are in blue. Compared with the results for the same configuration in Fig. 9, the distributions become fatter and have larger variance. The right tails have higher probabilities, due to the poorer tracking performance with the imperfect system models.

In the first EKF, the state estimation MSE L_{MSE}^G is used as a loss function. Its histograms are displayed in Fig. 10 in green. For both of the histograms, the distributions move to the left, which means the proposed training approach finds



(a) Histogram of State Estimation RMS Errors



(b) Histogram of Posterior Negative Log Likelihoods

Fig. 10. Histogram results for imperfect motion and measurement models.

better system noise covariance matrices compared with the EKF before training.

In the second EKF, the loss function is constructed by using the measurement residual L_{Res}^M . The histograms for the second EKF are showed in Fig. 10 in red. Compared with the original manually tuned EKF results, better tracking performance has been shown in terms of both the state estimation error and likelihood. However, compared with the results for the first EKF trained with the ground truth data, it loses the competition in terms of both of the evaluation metrics. Especially in Fig. 10(b), its histogram moves not as far as the first EKF trained with ground truth data. This is due to the fact that during training,

TABLE II
TESTING RESULTS. -LogL: NEGATIVE LOG-LIKELIHOOD

Loss	RMSE (State)	RMSE (meas)	-LogL (State)	-LogL (meas)
Manually Tuned	7.738	12.616	5.135	5.732
L_{MSE}^G	2.552	6.461	3.376	4.391
L_{Res}^M	2.605	6.473	3.412	4.632

the loss function is constructed based on model parameters that are far from their true values.

In Table II, the tracking performance in terms of the RMSE and likelihood are provided for EKFs trained with different loss functions. From the results, the EKF trained using the ground truth outperforms the one without by all the different criteria. One of the reasons is that without using the ground truth data, the training relies on the correct model assumption, and the use of imperfect models will cause larger degradation of the training performance.

V. CONCLUSION

In this paper, a method called EKFNet was proposed, which can fine-tune the EKF automatically with or without the ground truth data. For learning the best process and measurement noise covariance parameters offline, we constructed and used four different objective functions. The whole framework is trained using gradient descent, and the gradients are calculated based on backpropagation through time. In a tracking example using real GPS data, the proposed method was shown to significantly outperform an existing method and a manually tuned EKF.

As discussed in [16], the learning process may be sensitive to the initialization of the process and measurement noise covariances, and obtain locally optimal solutions. In our future work, we plan to develop a learning approach that is robust to different initialization values. It is well known that the EKF's performance depends on where the Taylor series expansion is performed. It will be an interesting topic to design a neural network that can jointly learn the measurement and processing noise parameters and a mechanism for selecting the expansion points.

APPENDIX A

BACK PROPAGATION FROM UPDATE MODULE TO PREDICTION MODULE

Suppose that $\mathbf{W} = \mathbf{A}\mathbf{X}$, where $\mathbf{W}$, $\mathbf{X}$, and $\mathbf{A}$ all have proper dimensions, and $a(\mathbf{X})$ is a scalar-valued function of $\mathbf{X}$. By expanding $\mathbf{W} = \mathbf{A}\mathbf{X}$ explicitly, it can be shown that

$$\frac{\partial a}{\partial \mathbf{X}} = \mathbf{A}^T \frac{\partial a}{\partial \mathbf{W}} \quad (57)$$

Similarly, if $\mathbf{Y} = \mathbf{B}\mathbf{X}^T$, then

$$\frac{\partial b}{\partial \mathbf{X}} = \left(\frac{\partial b}{\partial \mathbf{Y}} \right)^T \mathbf{B} \quad (58)$$

and if $\mathbf{Z} = \mathbf{X}\mathbf{C}$, then

$$\frac{\partial c}{\partial \mathbf{X}} = \left(\frac{\partial c}{\partial \mathbf{Z}} \right) \mathbf{C}^T \quad (59)$$

A. Derivation of (21)

The derivation of (21) is straightforward:

$$\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \hat{\mathbf{x}}_{k|k-1}} = \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \hat{\mathbf{x}}_{k|k}} = \frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k}} \quad (60)$$

The first step in (60) results from (6a). Note that in the loss function L defined in (14), l_k^G is the only term that is a function of $\hat{\mathbf{x}}_{k|k}$, which leads to the second step in (60).

B. Derivation of (22) and (23)

In the update module, based on (5) and (6b), both $\mathbf{K}_k$ and $\mathbf{P}_{k|k}$ are functions of $\mathbf{P}_{k|k-1}$. Therefore, using chain rules specified in (57) and (59), we can prove that

$$\begin{aligned} \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{P}_{k|k-1}} &= \frac{\partial L}{\partial l_k^G} \left[(\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T \frac{\partial l_k^G}{\partial \mathbf{P}_{k|k}} + \frac{\partial l_k^G}{\partial \mathbf{K}_k} \mathbf{S}_k^{-1} \mathbf{H}_k \right] \\ &= (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T \frac{\partial L}{\partial \mathbf{P}_{k|k}} + \frac{\partial L}{\partial \mathbf{K}_k} \mathbf{S}_k^{-1} \mathbf{H}_k \end{aligned} \quad (61)$$

where the second step follows from the fact that in the loss function L , only l_k^G is a function of $\mathbf{P}_{k|k}$ and $\mathbf{K}_k$. Now based on (6a) and (6b), and using chain rule (59) twice, it can be shown that

$$\frac{\partial L}{\partial \mathbf{K}_k} = \frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k}} \tilde{\mathbf{y}}_k^T - \frac{\partial L}{\partial \mathbf{P}_{k|k}} \mathbf{P}_{k|k-1} \mathbf{H}_k^T \quad (62)$$

APPENDIX B

BACK PROPAGATION FROM UPDATE MODULE TO MEASUREMENT PREDICTION MODULE

A. Derivation of (24)

With (6a) and chain rule (57), it is easy to show that

$$\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \tilde{\mathbf{y}}_k} = \mathbf{K}_k^T \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \hat{\mathbf{x}}_{k|k}} = \mathbf{K}_k^T \frac{\partial L}{\partial \hat{\mathbf{x}}_{k|k}} \quad (63)$$

where the second step is due to the fact that in the loss function L , only l_k^G is a term that is a function of $\hat{\mathbf{x}}_{k|k}$.

B. Derivation of (25)

From (5) and (6b), we know that both $\mathbf{K}_k$ and $\mathbf{P}_{k|k}$ are functions of $\mathbf{H}_k$. First, (5) can be re-written as

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} (\mathbf{S}_k^{-1} \mathbf{H}_k)^T = \mathbf{P}_{k|k-1} \mathbf{U}^T \quad (64)$$

where

$$\mathbf{U} \triangleq \mathbf{S}_k^{-1} \mathbf{H}_k \quad (65)$$

Using chain rule (58), we have

$$\frac{\partial l_k^G}{\partial \mathbf{U}} = \left(\frac{\partial l_k^G}{\partial \mathbf{K}_k} \right)^T \mathbf{P}_{k|k-1} \quad (66)$$

Likewise, (6b) can be written as

$$\mathbf{P}_{k|k} = \mathbf{P}_{k|k-1} - \mathbf{K}_k \mathbf{H}_k \mathbf{P}_{k|k-1} = \mathbf{P}_{k|k-1} - \mathbf{K}_k \mathbf{V} \quad (67)$$

where

$$\mathbf{V} \triangleq \mathbf{H}_k \mathbf{P}_{k|k-1} \quad (68)$$

Using chain rule (57), we obtain

$$\frac{\partial l_k^G}{\partial \mathbf{V}} = -\mathbf{K}_k^T \frac{\partial l_k^G}{\partial \mathbf{P}_{k|k}} \quad (69)$$

Now based on (65) and (68), and using chain rules (57) and (59) respectively, it can be shown that

$$\begin{aligned} \frac{\partial l_k^G}{\partial \mathbf{H}_k} &= \mathbf{S}_k^{-1} \frac{\partial l_k^G}{\partial \mathbf{U}} + \frac{\partial l_k^G}{\partial \mathbf{V}} \mathbf{P}_{k|k-1} \\ &= \mathbf{S}_k^{-1} \left(\frac{\partial l_k^G}{\partial \mathbf{K}_k} \right)^T \mathbf{P}_{k|k-1} - \mathbf{K}_k^T \frac{\partial l_k^G}{\partial \mathbf{P}_{k|k}} \mathbf{P}_{k|k-1} \end{aligned} \quad (70)$$

which leads to

$$\begin{aligned} \frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{H}_k} &= \mathbf{S}_k^{-1} \left(\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{K}_k} \right)^T \mathbf{P}_{k|k-1} \\ &\quad - \mathbf{K}_k^T \left(\frac{\partial L}{\partial l_k^G} \frac{\partial l_k^G}{\partial \mathbf{P}_{k|k}} \right) \mathbf{P}_{k|k-1} \\ &= \mathbf{S}_k^{-1} \left(\frac{\partial L}{\partial \mathbf{K}_k} \right)^T \mathbf{P}_{k|k-1} - \mathbf{K}_k^T \frac{\partial L}{\partial \mathbf{P}_{k|k}} \mathbf{P}_{k|k-1} \end{aligned} \quad (71)$$

in which the second step is due to the fact that in the loss function L , l_k^G is the only term that is a function of $\mathbf{P}_{k|k}$ and $\mathbf{K}_k$.

C. Derivation of (26)

(5) can be re-written as

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{S}_k^{-1} = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{T} \quad (72)$$

where

$$\mathbf{T} \triangleq \mathbf{S}_k^{-1} \quad (73)$$

Using chain rule (57), we have

$$\frac{\partial l_k^G}{\partial \mathbf{T}} = (\mathbf{P}_{k|k-1} \mathbf{H}_k^T)^T \frac{\partial l_k^G}{\partial \mathbf{K}_k} \quad (74)$$

Based on Lemma 1 of [48] or Section 2.2 of [49], and the definition of $\mathbf{T}$ in (73), it can be shown that

$$\begin{aligned} \frac{\partial l_k^G}{\partial \mathbf{S}_k} &= -(\mathbf{S}_k^{-1})^T \frac{\partial l_k^G}{\partial \mathbf{T}} (\mathbf{S}_k^{-1})^T \\ &= -\mathbf{S}_k^{-1} \mathbf{H}_k \mathbf{P}_{k|k-1} \frac{\partial l_k^G}{\partial \mathbf{K}_k} \mathbf{S}_k^{-1} \end{aligned} \quad (75)$$

With (75), and the fact that in the loss function L , l_k^G is the only term that depends on $\mathbf{K}_k$, (26) can be easily proved.

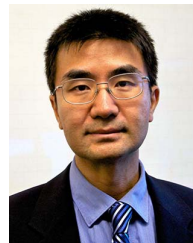
REFERENCES

- [1] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation: Theory Algorithms and Software*. Hoboken, NJ, USA: Wiley, 2004.
- [2] Y. Bar-Shalom, P. K. Willett, and X. Tian, *Tracking and Data Fusion: A Handbook of Algorithms*. Storrs, CT: YBS Publishing, 2011.
- [3] L. Xu, "Deep learning assisted intelligent visual and vehicle tracking systems," Ph.D. dissertation, Virginia Commonwealth Univ., Richmond, VA, USA, Aug. 2021.
- [4] S. J. Julier and J. K. Uhlmann, "Unscented filtering and nonlinear estimation," *Proc. IEEE*, vol. 92, no. 3, pp. 401–422, Mar. 2004.
- [5] B. Ristic, S. Arulampalam, and N. Gordon, *Beyond the Kalman Filter: Particle Filters for Tracking Applications (Artech House Radar Library)*. Norwood, MA, USA: Artech House, 2004.
- [6] L. Xu and R. Niu, "Semi-supervised visual tracking based on variational Siamese network," in *Proc. Int. Conf. Dyn. Data Driven Appl. Syst.*, in Lecture Notes in Computer Science, F. Darema, E. Blasch, S. Ravela, and A. Aved, Eds., vol. 12312, Cham, Switzerland: Springer, 2020, pp. 328–336.
- [7] L. Xu, R. Niu, and E. P. Blasch, "Uncertainty aware EKF: A tracking filter learning LiDAR measurement uncertainty," in *Proc. 25th Int. Conf. Inf. Fusion (FUSION)*, Piscataway, NJ, USA: IEEE Press, 2022.
- [8] L. Xu and R. Niu, "Tracking visual object as an extended target," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2021, pp. 664–668.
- [9] L. Xu and R. Niu, "TrafficEKF: A learning based traffic aware extended Kalman filter," in *Proc. IEEE 24th Int. Conf. Inf. Fusion (FUSION)*, 2021, pp. 1–8.
- [10] M. Pilté, S. Jouaber, S. Bonnabel, F. Barbaresco, M. Fragu, and N. Honoré, "Invariant extended Kalman filter applied to tracking for air traffic control," in *Proc. Int. Radar Conf. (RADAR)*, Sep. 2019.
- [11] S. T. Barratt and S. P. Boyd, "Fitting a Kalman smoother to data," in *Proc. Amer. Control Conf. (ACC)*, Jul. 2020, pp. 1526–1531.
- [12] P. Abbeel, A. Coates, M. Montemerlo, A. Y. Ng, and S. Thrun, "Discriminative training of Kalman filters," in *Proc. Robot.: Sci. Syst. I*, 2005.
- [13] D. M. Asmar and G. Eslinger, "Nonlinear programming approach to filter tuning," MIT Space Systems Laboratory, Cambridge, MA, USA, Tech. Rep., 2012. Available: <https://api.semanticscholar.org/CorpusID:3946639>
- [14] Z. Chen, C. Heckman, S. Julier, and N. Ahmed, "Weak in the NEES?: Auto-tuning Kalman filters with Bayesian optimization," in *Proc. 21st Int. Conf. Inf. Fusion (FUSION)*, 2018, pp. 1072–1079.
- [15] K. V. Yuen, P. F. Liang, and S. C. Kuok, "Online estimation of noise parameters for Kalman filter," *Structural Eng. Mech.*, vol. 47, no. 3, pp. 361–381, 2013.
- [16] K. V. Yuen and S. C. Kuok, "Online updating and uncertainty quantification using nonstationary output-only measurement," *Mech. Syst. Signal Process.*, vols. 66–67, pp. 62–77, Jan. 2016.
- [17] L. Zhang, D. Sidoti, A. Bienkowski, K. R. Pattipati, Y. Bar-Shalom, and D. L. Kleinman, "On the identification of noise covariances and adaptive Kalman filtering: A new look at a 50 year-old problem," *IEEE Access*, vol. 8, pp. 59362–59388, 2020.
- [18] S. Jouaber, "Utilisation de méthodes d'apprentissage automatique pour des tâches de pistage radar et de classification d'aéronefs," Ph.D. dissertation, Université Paris Sciences et Lettres, Paris, France, Sep. 2022.
- [19] S. Jouaber, S. Bonnabel, S. Velasco-Forero, M. Pilté, and J. Angulo, "Real-time classification of aircrafts manoeuvres," *J. Signal Process. Syst.*, vol. 95, pp. 425–434, Apr. 2023.
- [20] P. Ondruska and I. Posner, "Deep tracking: Seeing beyond seeing using recurrent neural networks," 2016, *arXiv:1602.00991*.
- [21] R. Wilson and L. Finkel, "A neural implementation of the Kalman filter," in *Proc. Adv. Neural Inf. Process. Syst.*, 2009, pp. 2062–2070.
- [22] T. Haarnoja, A. Ajay, S. Levine, and P. Abbeel, "Backprop KF: Learning discriminative deterministic state estimators," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 4376–4384.
- [23] H. Coskun, F. Achilles, R. DiPietro, N. Navab, and F. Tombari, "Long short-term memory Kalman filters: Recurrent neural estimators for pose regularization," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 5524–5532.
- [24] Y. Iiguni, H. Sakai, and H. Tokumaru, "A real-time learning algorithm for a multilayered neural network based on the extended Kalman filter," *IEEE Trans. Signal Process.*, vol. 40, no. 4, pp. 959–966, Apr. 1992.
- [25] S. Haykin, *Kalman Filtering and Neural Networks*. Hoboken, NJ, USA: Wiley, 2001.

- [26] R. Togneri and L. Deng, "Joint state and parameter estimation for a target-directed nonlinear dynamic system model," *IEEE Trans. Signal Process.*, vol. 51, no. 12, pp. 3061–3070, Dec. 2003.
- [27] N. M. Vural, S. Ergüt, and S. S. Kozat, "An efficient and effective second-order training algorithm for LSTM-based adaptive learning," *IEEE Trans. Signal Process.*, vol. 69, pp. 2541–2554, 2021.
- [28] A. Kloss, G. Martius, and J. Bohg, "How to train your differentiable filter," *Auton. Robots*, vol. 45, pp. 561–578, May 2021.
- [29] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. G. van Sloun, and Y. C. Eldar, "KalmanNet: Neural network aided Kalman filtering for partially known dynamics," *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.
- [30] S. Jouaber, S. Bonnabel, S. Velasco-Forero, and M. Pilté, "NNAKF: A neural network adapted Kalman filter for target tracking," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, Jun. 2021, pp. 4075–4079.
- [31] S. Jouaber, S. Bonnabel, M. Pilté, and S. Velasco-Forero, "Neural network adapted Kalman filter," in *Proc. Int. Conf. Integr. Air Missile Defence (IAMD)*, Dec. 2021.
- [32] R. Jonschkowski, D. Rastogi, and O. Brock, "Differentiable particle filters: End-to-end learning with algorithmic priors," 2018, *arXiv:1805.11122*.
- [33] P. Karkus, D. Hsu, and W. S. Lee, "Particle filter networks with application to visual localization," in *Proc. Conf. Robot Learn.*, PMLR, 2018, pp. 169–178.
- [34] X. Ma, P. Karkus, D. Hsu, and W. S. Lee, "Particle filter recurrent neural networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 4, 2020, pp. 5101–5108.
- [35] A. Corenflos, J. Thornton, G. Deligiannidis, and A. Doucet, "Differentiable particle filtering via entropy-regularized optimal transport," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 2100–2111.
- [36] H. Wen, X. Chen, G. Papagiannis, C. Hu, and Y. Li, "End-to-end semi-supervised learning for differentiable particle filters," 2020, *arXiv:2011.05748*.
- [37] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proc. IEEE*, vol. 78, no. 10, pp. 1550–1560, Oct. 1990.
- [38] L. Xu and R. Niu, "EKFNet: Learning system noise statistics from measurement data," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2021, pp. 4560–4564.
- [39] S. Ruder, "An overview of gradient descent optimization algorithms," 2016, *arXiv:1609.04747*.
- [40] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [41] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [42] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [43] J. Almeida and V. M. Santos, "Real time egomotion of a nonholonomic vehicle using LIDAR measurements," *J. Field Robot.*, vol. 30, no. 1, pp. 129–141, 2013.
- [44] X. R. Li and V. P. Jilkov, "Survey of maneuvering target tracking. Part I. Dynamic models," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 39, no. 4, pp. 1333–1364, Oct. 2003.
- [45] R. Schubert, C. Adam, M. Obst, N. Mattern, V. Leonhardt, and G. Wanielik, "Empirical evaluation of vehicular models for ego motion estimation," in *Proc. IEEE Intell. Veh. Symp. (IV)*, 2011, pp. 534–539.
- [46] J. P. Laumond, Ed., *Robot Motion Planning and Control*, vol. 229. Berlin, Heidelberg, Germany: Springer, 1998.
- [47] R. Schubert, E. Richter, and G. Wanielik, "Comparison and evaluation of advanced motion models for vehicle tracking," in *Proc. 11th Int. Conf. Inf. Fusion*, Piscataway, NJ, USA: IEEE Press, 2008.
- [48] Z. He, S. Xie, S. Ding, and A. Cichocki, "Convolutional blind source separation in the frequency domain based on sparse representation," *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 15, no. 5, pp. 1551–1563, Jul. 2007.
- [49] K. B. Petersen and M. S. Pedersen, "The matrix cookbook." version 20121115. Accessed: Aug. 16, 2020. [Online]. Available: <http://www2.compute.dtu.dk/pubdb/pubs/3274-full.html>



Liang Xu received the master's degree in computer science from Georgia Institute of Technology, the master's degree in mechanical engineering from Wichita State University, and the Ph.D. degree in computer engineering from Virginia Commonwealth University, in 2021, where he developed an intelligent vehicle tracking and fusion system. Currently, he is a Senior Research Scientist with the Motional, Inc., where he designs and implements end-to-end machine learning tracking systems for autonomous vehicles. His research interests include optimizing real-time sensor fusion systems, integrating vision and LiDAR data, and improving perception robustness and safety.



Ruixin Niu (Senior Member, IEEE) received the Ph.D. degree in electrical engineering from the University of Connecticut, in 2001. He is currently an Associate Professor with the Department of Electrical and Computer Engineering, Virginia Commonwealth University (VCU). Before joining VCU, he was a Research Assistant Professor with Syracuse University. He has more than 130 publications in the above-mentioned areas. His research interests include statistical signal processing and its applications, including detection, estimation, information fusion, sensor networks, communications, and machine learning. He received the Best Paper Award at the International Conference on Information Fusion (Fusion Conference), in 2004. He is a co-author of the paper that won the Best Student Paper Award at the Fusion Conference, in 2010. He serves as an Associate Editor of IEEE SIGNAL PROCESSING LETTERS, and an Area Editor of the *Journal of Advances in Information Fusion*. He was an Associate Editor of the *International Journal of Distributed Sensor Networks*, from 2010 to 2012, and an Associate Editor of IEEE TRANSACTIONS ON SIGNAL PROCESSING, from 2012 to 2016.